

CAST: Testing the Attribution Stability of AI-Generated-Code Detectors Under Controlled Source Transformations

Anonymous Author(s)

Abstract

AI-generated-code detectors are evaluated almost exclusively on static snapshots, even though real code is routinely reformatted, renamed, and restructured after creation — for example, by an IDE on save. This is a problem with real stakes: a detector’s verdict can feed an academic-integrity case, a hiring assessment, or a project’s AI-disclosure policy, and a verdict that changes under a cosmetic edit is not measuring origin, it is measuring the snapshot. We present CAST (Code Attribution Stability Testing), a controlled, distance-instrumented methodology that measures whether a detector’s human-versus-AI attribution survives routine, non-adversarial transformations, and uses a canonicalization intervention to diagnose whether any instability found is presentation-driven.

Evaluating three independently obtained detectors on 199 held-out Python and Java programs from the Droid corpus, we find that reformatting with a standard tool (black/google-java-format) flips a reconstructed detector’s (DroidDetect-Base) correct verdict on genuinely human-written code to “AI” 58.0% of the time (29/50 baseline-correct human samples), while leaving its verdict on actually AI-authored code almost unchanged (0.0–4.3%). A reverse experiment — perturbing AI-authored code toward human-like “messiness” — induces the symmetric AI-to-human flip in only 5.2% of cases (5/96), confirming the asymmetry is directional, though the two perturbations are not intensity-matched. This pattern is consistent with the detector having learned tidy formatting as a proxy for AI authorship, and constitutes a genuine mislabel under the Droid corpus’s own definitions, not a defensible edge case. The other two detectors illustrate a methodological hazard rather than a finding of their own: LLMSniffer’s low error rate is not evidence of robustness, since its own accuracy on this corpus sits at or below a majority-class baseline (67.0% Python, 44.4% Java); and DetectCodeGPT’s broad score movement cannot be distinguished from its own re-scoring noise, which an identity-transformation control measures directly (a 30.0% decision-flip rate on *unchanged* input). Canonicalization attenuates DroidDetect-Base’s residual formatting-induced movement by 14.8–15.4×, but canonicalized input is itself frequently out-of-distribution for the detector, so this diagnostic does not yet establish a practical mitigation. Static accuracy alone therefore does not characterize detector reliability under routine code evolution, and a low instability rate does not, by itself, establish robustness.

Keywords

AI-generated code detection, robustness testing, metamorphic testing, software evolution, code authorship attribution

1 Introduction

AI coding assistants now write a substantial share of the code developers review, edit, and ship. Detectors that attribute this code to a human or AI origin are typically evaluated the same way a

classifier is: on a fixed test set, scored once, with accuracy as the verdict. That verdict is not academic. Detectors are already being proposed for academic-integrity enforcement in classrooms, screening in take-home hiring assessments, and compliance with project- or organization-level AI-disclosure policies. A false accusation in any of these settings has a real person on the other end of it.

1.1 Motivation

The standard evaluation protocol quietly assumes the code being judged stays put. It does not. An IDE reformats a file on save, a refactor renames a variable, a reviewer rewrites a loop — none of it changes what the program does, yet any of it can change what a detector says about who wrote it. A detector whose verdict flips under a cosmetic edit is not measuring origin; it is measuring the snapshot.

This is a routine, non-adversarial threat, and it is distinct from the adversarial setting the closest prior work targets. Robustness studies for AI-code detectors vary how the code is generated in the first place [11], not how it changes afterward. Detector papers that do report a robustness number typically report it for one perturbation at most, with no stratified account of which kinds of edits matter and why. Work explicitly adversarial to detectors trains a classifier against an attacker’s renaming strategy [6] or evaluates under deliberately adversarial generation and rewriting scenarios [1]. Separate lines of work show that renaming alone can defeat a code watermark [7], and that metamorphic testing of code models more broadly treats identifier renaming as a standard, non-adversarial transformation [8]. None of this literature measures whether an already-trained, unmodified AI-code detector’s verdict survives a non-adversarial edit that a developer or a tool would make anyway, or reports which specific kinds of edits matter and why.

1.2 Objective

This paper’s objective is to make attribution stability an explicit, measurable property of an AI-code detector, not an afterthought to accuracy, and to build the smallest instrument that can measure it. We introduce **CAST** (Code Attribution Stability Testing): given a program of known origin, CAST applies a validated transformation, measures the resulting change at the text, token, and AST level, and records how the detector’s score and decision move. Where movement is large, CAST adds a second, targeted step: canonicalizing both sides of the pair to test whether the instability is a presentation artifact or something deeper.

Applied to three detectors over 199 held-out Python and Java programs [1], this approach exposes what a single accuracy number cannot: one detector shows a large, class-specific, false-accusing instability under formatting that a canonicalization intervention only partially and ambiguously attenuates; the other two detectors’

apparently low instability turns out to be confounded by degenerate baseline accuracy and by re-scoring noise, respectively — a finding about evaluation practice in its own right, not just about the detectors themselves.

1.3 Contributions

This paper makes two contributions.

- **CAST**, a controlled, distance-instrumented diagnostic methodology for evaluating the attribution stability of already-trained AI-code detectors under routine, non-adversarial edits. CAST’s canonicalization intervention and identity-transformation noise-floor control exist specifically to prevent the two failure modes this paper’s own results expose: an apparent stability finding confounded by degenerate baseline accuracy, and one confounded by re-scoring noise. A low Induced Error Rate is uninterpretable without both checks, and this paper demonstrates why on two of its own three evaluated detectors, not as a separate lesson but as CAST’s methodology doing its job.
- An empirical result: a reconstructed but accuracy-validated detector (DroidDetect-Base) shows a large, reproducible, class-specific, and directionally asymmetric instability under reformatting — flipping correct human-code verdicts to “AI” far more often than the reverse (Fisher’s exact $p < 10^{-11}$) — consistent with the detector having learned tidy formatting as an AI-authorship proxy.

The remainder of the paper is organized as follows. Section 2 defines CAST and the canonicalization intervention. Section 3 describes the dataset, transformations, detectors, and research questions. Section 4 reports results. Section 5 discusses their implications. Section 6 lists threats to validity. Section 7 positions CAST against related work. Section 8 outlines future plans, and Section 9 concludes.

2 CAST

2.1 Attribution stability

Attribution stability is the degree to which a detector’s output for a program is preserved under routine, non-adversarial edits — distinct from adversarial robustness, which studies a worst-case attacker searching for an evasive rewrite. CAST’s transformations are fixed, deterministic operators applied uniformly across the corpus, not an adversarial search against any particular detector. CAST treats score movement and decision-flip rate as first-class observations, rather than reducing the evaluation to a single post-transformation accuracy number.

2.2 Methodology

For a detector D , a transformation T , and a program C_0 of known provenance, CAST proceeds in five steps, illustrated in Figure 1.

- (1) Apply T to obtain $T(C_0)$.
- (2) Validate that $T(C_0)$ is well-formed at or above C_0 ’s own syntactic-validity tier. Non-applicable and invalid outcomes are recorded, not discarded, so that applicability itself is a reportable quantity.

- (3) Measure d_{text} (character-level edit distance), d_{token} (edit distance over the language’s own lexical token stream — Python’s tokenize output including INDENT/DEDENT, or Java tree-sitter leaves), and d_{AST} (AST node-count change), so that the magnitude of an edit can later be related to the magnitude of a detector’s response.
- (4) Score $D(C_0)$ and $D(T(C_0))$.
- (5) Aggregate, per detector, language, and transformation family, the mean and median score movement $\Delta_D = D(T(C_0)) - D(C_0)$, the decision-flip rate, and the **Induced Error Rate**, defined as:

$$\text{IER} = \frac{\#\{\text{baseline-correct programs whose decision flips after } T\}}{\#\{\text{baseline-correct programs}\}} \quad (1)$$

IER is conditioned on the detector having been correct on C_0 . A flip on a program the detector already misclassified would otherwise conflate ordinary detector error with transformation-induced instability, understating the difference between a detector that is simply inaccurate and one that is accurate but unstable. Detectors also expose incompatible score semantics across implementations: a calibrated probability, an uncalibrated sigmoid output, or an unnormalized statistic with no fixed range. CAST therefore does not compare raw $|\Delta_D|$ across detectors. IER, a bounded proportion with a consistent interpretation regardless of the underlying score scale, is the primary cross-detector metric; raw score movement is interpreted only within a given detector’s own distribution, never compared across detectors directly.

2.3 Identity-transformation noise floor

Some detectors are not deterministic at inference time: a statistic computed from a sampled or randomly perturbed process can, in principle, return a different verdict on literally unchanged input simply because the random seed differed. Before attributing any instability to the transformation itself, CAST re-scores C_0 against itself with a fresh random seed and no transformation applied, and reports the resulting decision-flip rate and mean $|\Delta|$ as a noise floor. A detector’s transformation-induced IER is only interpretable relative to this floor: an IER within the range of the noise floor cannot be distinguished from measurement noise, no matter how elevated it looks in isolation. Deterministic detectors (no sampling at inference) have a zero noise floor by construction, and this control is skipped for them.

2.4 Canonicalization intervention

When a transformation family produces large score movement, CAST additionally scores $F(C_0)$ and $F(T(C_0))$ for a deterministic canonicalizer F that removes a specific, named category of variation — in this study, whitespace, blank lines, comments, and quote style, implemented by re-tokenizing the program and rejoining its semantic tokens. A pair for which $F(C_0) = F(T(C_0))$ collapses to $\Delta = 0$ by construction. This subset is reported separately from the residual pairs for which a genuine difference remains after canonicalization, so that the diagnostic result is not an artifact of the trivially-resolved cases inflating an average. A substantial reduction in the residual subset’s score movement constitutes intervention evidence that the removed category of variation contributes to the

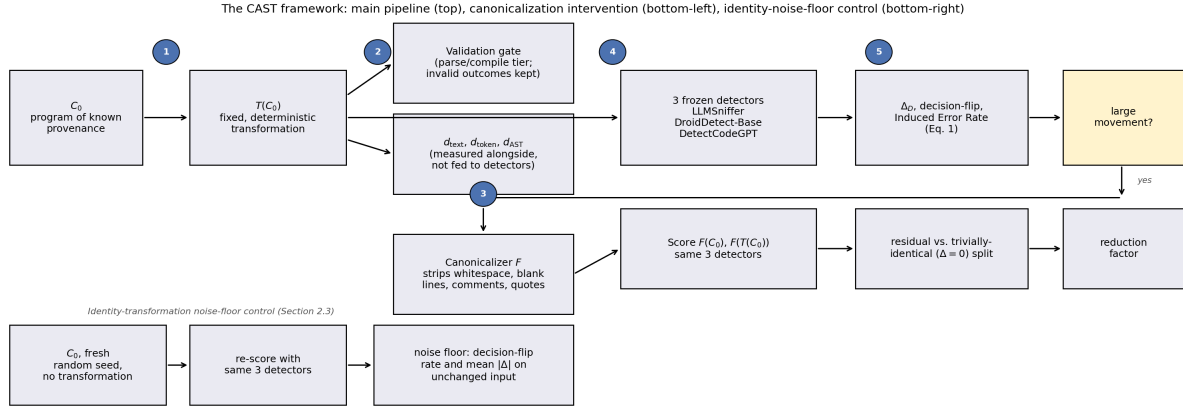


Figure 1: The CAST framework. Top: the main pipeline – a transformation T is applied to a program C_0 of known provenance (1), validated (2) and distance-measured (3) against C_0 , then both C_0 and $T(C_0)$ are scored by all three detectors (4) to compute score movement and the Induced Error Rate (5). Bottom-left: when movement is large, the canonicalization intervention scores a presentation-stripped version of both sides to isolate presentation-driven instability. Bottom-right: the identity-transformation noise-floor control re-scores C_0 against itself with a fresh seed and no transformation, establishing how much decision-flip rate is measurement noise rather than a real transformation effect.

observed instability. It does not constitute proof of a specific internal mechanism, and, as this paper’s own results show (Section 4.2), it can be confounded by the canonicalized input itself becoming out-of-distribution for the detector under test – a limitation of this particular canonicalizer, not of the diagnostic logic in general.

3 Study Design

3.1 Dataset

This evaluation uses DroidCollection’s [1] held-out test split: 199 programs (100 Python, 99 Java), stratified across the corpus’s three provenance labels, HUMAN_GENERATED, MACHINE_GENERATED, and MACHINE_REFINED. MACHINE_REFINED denotes human-written code that has been modified or extended by a language model – a collaborative, human-plus-LLM category distinct from either pure human authorship or pure machine generation. This distinction matters directly for interpreting one of this paper’s results (Section 4.1).

3.2 Transformations

Three families are treated as confirmatory: formatting (black for Python, google-java-format for Java), lexical renaming of one identifier, and control-flow rewriting (one canonical loop-form rewrite). Applicability rate across these three families was 0.616; transformation success rate among applicable attempts was 0.992. A fourth stage attempted two deeper transformation families, local structural extraction and semantics-preserving rewriting, at two intensities each; this stage is reported as a feasibility finding (Section 4.3), not a confirmatory one, because of the corpus-composition limitation described there.

3.3 Validation

Python validity is checked via `compile()`. Java is tiered, given the corpus’s substantial share of function- and class-level fragments rather than complete compilation units: STANDALONE_VALID > WRAPPER_VALID (compiles only inside a synthetic harness, used as a validation instrument and never itself scored) > STRUCTURAL_ONLY > PARSE_INVALID.

3.4 Detectors

Of five targeted detectors, two had a released checkpoint runnable as-is; a third, DroidDetect-Base, required reconstructing its architecture from the checkpoint alone, since no modeling code was published for it.

LLMSniffer [2] (GraphCodeBERT [19] with a supervised-contrastive head) reaches 90.0% sanity-check accuracy on GPTSniffer’s own Java test subset [3], but only 67.0% (Python) and 44.4% (Java) on this study’s 199-program baseline – at or below a majority-class baseline of 67.0%/66.7% respectively (Section 6). This gap between its published in-domain accuracy and its out-of-domain accuracy on Droid is itself informative: a detector’s reported accuracy figure does not transfer automatically to a new corpus, and this paper’s methodological point – that a low IER needs a baseline-accuracy check – would not have been visible without measuring it directly on the same corpus used for the stability evaluation.

DroidDetect-Base (ModernBERT-base [18] with a projection layer and a four-class classification head) ships a public checkpoint with no accompanying modeling code. We reconstructed its architecture from the checkpoint’s own parameter names and shapes, and validated the reconstruction against a disjoint dev-split subset ($n=24$), reaching 87.5% accuracy (Wilson 95% CI [69.0, 95.7]). This interval does not fully contain the 98–99% accuracy the same reconstruction reaches on the held-out test split used throughout the

rest of this study (Section 4.1). We cannot yet fully rule out that this reflects a genuine reconstruction gap rather than dev-subset difficulty, and treat this as an open validation question (Section 6); we note, though, that a fundamentally incorrect reconstruction of a 4-class head would be expected to perform near the 25% chance level, not 87.5%, making a substantially correct implementation more consistent with the dev-split result than a severely broken one; $n=24$ also genuinely limits precision here (each additional correct call shifts accuracy by 4.2 percentage points), so the interval’s coarseness reflects sample size as much as it reflects disagreement. Because this paper’s central empirical finding rests entirely on this one reconstructed detector, this is the single most important threat to the paper’s soundness, and future work (Section 8) prioritizes closing it directly.

DetectCodeGPT [4] is a zero-shot statistic from the DetectGPT [14]/NPR [15] family, computed by perturbing the input with masked-span fill-in and comparing log-likelihoods under CodeLlama-7B [16] and CodeT5p-770M [17]. A pre-registered convergence study established that no perturbation count $K < 50$ satisfies a predeclared convergence criterion against the $K=50$ reference, so $K=50$ perturbations are used throughout. Its decision threshold, calibrated on an independent 40-sample set, reaches only 60.0% accuracy — near chance — so its raw score movement is treated as the primary evidence in this study, and its threshold-derived binary decisions as secondary, lower-confidence evidence.

GPTSniffer [3] and **CodeGPTSensor/+** [5] could not be integrated into this study, as neither released a pretrained checkpoint at the time of writing. We deliberately did not train replacements from scratch, since doing so would introduce confounds (training-data composition, hyperparameter choices, our own implementation bugs) distinct from evaluating a detector as its own authors published and validated it.

3.5 Research Questions

This evaluation is structured around three research questions.

- **RQ1:** How stable are detector scores and decisions under formatting, lexical renaming, and control-flow rewriting?
- **RQ2:** Do detectors exhibit similar or detector-specific stability profiles under identical transformations, and are apparent differences genuine or artifacts of baseline accuracy or measurement noise?
- **RQ3:** Does canonical normalization attenuate transformation-associated instability, and does this differ across detectors?

4 Results

4.1 RQ1/RQ2: detector-specific, transformation-specific profiles

Table 2 reports IER with baseline-correct denominators, for all three detectors across all three confirmatory transformation families and both languages. DroidDetect-Base’s formatting IER exceeds its lexical-rename and control-flow IER in both languages, collapsing to at or near 0% under both of the other transformations in Python. Wilson [20] 95% confidence intervals for formatting do not overlap those of the other two families in Python; Java’s smaller post-collapse denominators give wider, overlapping intervals, so the

effect is directionally consistent in Java but less sharply statistically separated there than in Python.

Table 1 breaks the formatting result down further, by the program’s *true* origin class rather than pooling across all AI-authored code: DroidDetect-Base’s formatting IER is 58.0% (29/50) on genuinely human-written code, but only 0.0% (0/58) on machine-generated code and 4.3% (2/47) on machine-refined code — a pattern that holds consistently when Python and Java are examined separately (Python: 66.7%/0.0%/6.5%; Java: 41.2%/0.0%/0.0%). This class-specific asymmetry is not a no-op artifact of the transformation simply doing nothing to already-tidy AI code: only 5–6% of AI-class formatting transformations leave the code byte-identical to its input.

To test this asymmetry more directly, we ran a reverse experiment: messifying AI-class code (introducing trailing whitespace, extra blank lines, and inconsistent quoting and spacing — a deterministic, semantics-preserving, validity-checked transformation applied across all 133 AI-class manifest rows, of which 106 were baseline-correct before messification and 96 remained after excluding 10 whose messified output failed a validity check) and measuring the reverse, AI-to-human flip rate. This induces a flip in only 5/96 cases (5.2%, 95% CI [2.2, 11.6]; Python 6.5%, Java 4.0%) — far below the 58.0% human-to-AI rate. This is not, however, an intensity-matched comparison: messify’s mean length-normalized edit size ($d_{\text{text}}/\text{len} = 0.064$) is smaller than formatting’s on human code (≈ 0.14), so the 58.0%/5.2% gap should be read as confirming the asymmetry is real and directional — ruling out a zero-sensitivity null result for AI code entirely — rather than as a precisely intensity-matched effect-size comparison (Section 6). Raw edit distance does not otherwise cleanly separate flipped from unflipped human samples once normalized by program length: flipped human samples have a smaller raw mean d_{text} (81.1 vs. 470.5 for unflipped samples), but this tracks their shorter mean length (650 vs. 3531 characters) rather than a proportionally smaller edit; length-normalized $d_{\text{text}}/\text{len}$ is comparable between the two groups (0.146 vs. 0.136), so program length remains an unexcluded candidate confound distinct from, and in addition to, the class-level asymmetry established above.

Score movement correlates only weakly with edit distance: $|\Delta|$ has a Spearman correlation of $\rho=0.22$ with d_{text} for DroidDetect-Base and $\rho=0.30$ for LLMSniffer, but essentially none with d_{AST} ($\rho=-0.20$ and 0.03 respectively) — consistent with a presentation-driven rather than structure-driven effect, since formatting changes text and token structure substantially while barely touching the AST.

Of the 29 flipped human samples for DroidDetect-Base, the underlying four-class output lands mostly on MACHINE_REFINED (17/29) rather than MACHINE_GENERATED (11/29), with one landing on MACHINE_GENERATED_ADVERSARIAL. As Section 3.1 notes, Droid defines MACHINE_REFINED specifically as human-written code that has been modified by an LLM. A deterministic code formatter is not an LLM, does not perform semantic refinement, and does not use any learned model at all; predicting MACHINE_REFINED for merely reformatted human code is therefore a genuine mislabel under Droid’s own construct definitions, not a defensible edge case reflecting some literal sense in which the formatter has “touched” the file.

Table 1: DroidDetect-Base formatting IER by true origin class (human-vs-AI collapse, pooled across languages).

True class	n (baseline-correct)	IER	95% CI
HUMAN_GENERATED	50	58.0% (29/50)	[44.2,70.6]
MACHINE_GENERATED	58	0.0% (0/58)	[0.0,6.2]
MACHINE_REFINED	47	4.3% (2/47)	[1.2,14.3]

LLMSniffer’s low IER (below 8.3% throughout; full per-family results in Table 2) is not, by itself, evidence of robustness: its own baseline accuracy on this corpus is 67.0% in Python (exactly its majority-class baseline, i.e., indistinguishable from a constant predictor) and 44.4% in Java (below its 66.7% majority-class baseline, i.e., anti-correlated with the true label more often than not). A detector with no signal, or negative signal, on a corpus will trivially show a low decision-flip rate simply because it is not tracking the input’s content in the first place.

DetectCodeGPT’s IER is elevated broadly across all three families (16.7–50.0%, secondary evidence given its weak calibration), peaking under control-flow rather than formatting — a diffuse pattern with no clear family-specific localization. An identity-transformation control (Section 2.3), re-scoring 40 frozen dev-split samples with an unchanged input but a freshly seeded perturbation set, found a comparably sized 30.0% decision-flip rate and a mean $|\Delta|$ of 0.925 — larger than the transformation-induced movement reported in Table 3. DetectCodeGPT’s elevated IER therefore cannot currently be distinguished from its own re-scoring noise.

The 58.0% (29/50) human-to-AI rate and the 5.2% (5/96) AI-to-human rate come from independent samples (different programs), so we test the asymmetry with Fisher’s exact test rather than a paired test: the difference is extreme (odds ratio 25.1, $p=2.1\times 10^{-12}$), confirming the asymmetry is not attributable to chance even setting aside the non-overlapping confidence intervals already reported.

Answer to RQ1/RQ2: DroidDetect-Base’s instability is real, class-specific, directionally asymmetric (58.0% vs. 5.2%, Fisher’s exact $p<10^{-11}$, though not an intensity-matched comparison), and overwhelmingly false-accusing under Droid’s own label definitions. The other two detectors’ apparently low instability is confounded: LLMSniffer’s by baseline accuracy at or below chance, and DetectCodeGPT’s by re-scoring noise of comparable magnitude to its reported effect. Stability is therefore detector-specific in a stronger sense than a simple ranking: two of the three “low-IER” outcomes in this study are not interpretable as stability claims at all without the accompanying controls.

4.2 RQ3: canonicalization intervention

The canonicalization diagnostic was applied to all 158 formatting pairs (94 Python, 64 Java). Canonicalizing both sides collapsed 108/158 (68.4%) to byte-identical input ($\Delta=0$ by construction); the remaining 50/158 retained a genuine residual difference and were rescored. Table 3 reports both the full-set and residual-only figures, so that the diagnostic’s real contribution is isolated from the trivially-resolved majority of pairs.

On the residual-only subset, DroidDetect-Base’s mean score movement still falls 14.8–15.4 $\times$ after canonicalization. This reduction, however, is complicated by an out-of-distribution effect that

this paper measured directly rather than assumed away: on the 100 canonicalized residual scores, DroidDetect-Base’s mean probability assigned to the *true* class drops to 0.223, with 66% of these scores below 0.05, against a 98–99% raw baseline accuracy on unmodified input. Canonicalized input is therefore often confidently *misclassified* by this detector, meaning the large apparent reduction in score movement may partly reflect the model failing in a different, more uniform way on out-of-distribution input, rather than genuinely recognizing the code as origin-preserved once presentation differences are removed. DetectCodeGPT shows no comparable reduction on the residual subset (Python 1.0 $\times$, i.e. essentially unchanged; Java 0.7 $\times$, i.e. an *increase* in movement, from a mean $|\Delta|$ of 0.566 to 0.866), and its canonicalized scores skew toward the AI decision (68% above its calibrated threshold, versus 50–54% on unmodified baseline input) — a further sign of distributional shift under canonicalization, not evidence against instability. LLMSniffer shows a smaller, genuinely real reduction on an already low baseline, with no comparable saturation signature. Figure 2 reports the same residual-only pooled reduction factors (3.6 $\times$ /14.9 $\times$ /0.9 $\times$ for LLMSniffer/DroidDetect-Base/DetectCodeGPT respectively), deliberately excluding the full-set numbers that the 68% of trivially-collapsed pairs would otherwise inflate.

Cohen’s d_z [21] is reported in the accompanying artifact but is not used as a headline statistic in this paper, since it standardizes by variance and can appear misleadingly large even for a practically small absolute movement — for example, LLMSniffer’s d_z ranges 1.39–1.49 despite a mean movement below 0.025 on its own bounded scale.

Answer to RQ3: This particular canonicalizer cannot cleanly answer RQ3 for DroidDetect-Base: canonicalized input is itself frequently misclassified, so the large residual-movement reduction is confounded with a saturation/out-of-distribution effect rather than being clean evidence of resolved instability. A layout-preserving normalizer, which keeps canonicalized input closer to the detector’s training distribution, is needed before this question can be settled with confidence. DetectCodeGPT’s movement is unreduced by canonicalization, and on Java, larger afterward — evidence against, not for, a presentation-driven explanation of its instability.

4.3 Corpus feasibility for deeper transformations

Local structural extraction and semantics-preserving rewriting were additionally attempted at two intensities each, as a feasibility probe for deeper transformation families beyond the three confirmatory ones. Of 796 attempted (language, family, intensity) combinations, only 37 (4.65%) were applicable at all, of which 35 validated successfully; only 4 of those 35 received true differential-execution verification (i.e., actually running the program before and after the transformation and comparing outputs), with the remaining 31 degrading to structural-only validation for lack of an unambiguous, stdin-free entry point to execute. This outcome directly reflects Droid’s composition: a substantial share of its rows are function- or class-level code fragments rather than complete, independently executable programs, which sharply bounds how densely this evaluation style currently extends to deeper, more realistic transformations. The resulting per-detector cells ($n=1-12$)

Table 2: Induced Error Rate by detector, language, and transformation family (n = baseline-correct denominator; $\dagger$ = secondary evidence given DetectCodeGPT’s weak calibration; DroidDetect-Base collapsed to human-vs-AI decision, Section 3.4).

Detector	Language	Family	IER (n)	95% CI
DroidDetect-Base	Python	formatting	26.1% (24/92)	[18.2,35.9]
DroidDetect-Base	Java	formatting	11.1% (7/63)	[5.5,21.2]
DroidDetect-Base	Python	lexical rename	0.0% (0/80)	[0.0,4.6]
DroidDetect-Base	Java	lexical rename	1.2% (1/82)	[0.2,6.6]
DroidDetect-Base	Python	control flow	0.0% (0/26)	[0.0,12.9]
DroidDetect-Base	Java	control flow	6.2% (1/16)	[1.1,28.3]
LLMSniffer	Java	formatting	8.3% (2/24)	[2.3,25.8]
LLMSniffer	Python	formatting	6.5% (4/62)	[2.5,15.4]
LLMSniffer	Python	lexical rename	1.9% (1/52)	[0.3,10.1]
LLMSniffer	Java	lexical rename	0.0% (0/35)	[0.0,9.9]
LLMSniffer	Python	control flow	0.0% (0/15)	[0.0,20.4]
LLMSniffer	Java	control flow	0.0% (0/4)	[0.0,49.0]
DetectCodeGPT $\dagger$	Python	control flow	50.0% (7/14)	[26.8,73.2]
DetectCodeGPT $\dagger$	Python	lexical rename	30.2% (13/43)	[18.6,45.1]
DetectCodeGPT $\dagger$	Python	formatting	25.0% (12/48)	[14.9,38.8]
DetectCodeGPT $\dagger$	Java	formatting	16.7% (6/36)	[7.9,31.9]
DetectCodeGPT $\dagger$	Java	lexical rename	16.7% (8/48)	[8.7,29.6]
DetectCodeGPT $\dagger$	Java	control flow	18.2% (2/11)	[5.1,47.7]

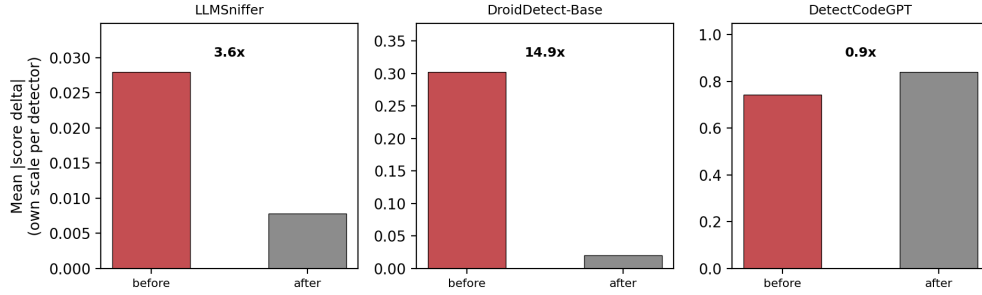


Figure 2: Canonicalization reduction factor, residual pairs only, per detector. Each panel uses its own y -axis; the factors are not cross-comparable, only within a detector’s own scale.

Table 3: Mean $|\Delta|$ before/after canonicalization, by language (raw scores; not comparable across detectors, only within).

Detector	Lang.	Full set		Residual-only	
		before	after (red.)	before	after (red.)
LLMSniffer	Py (94/32)	0.027	0.003 (8.6 $\times$)	0.032	0.009 (3.5 $\times$)
LLMSniffer	Java (64/18)	0.025	0.002 (16.7 $\times$)	0.020	0.005 (3.7 $\times$)
DroidDetect-Base	Py (94/32)	0.315	0.009 (34.9 $\times$)	0.394	0.027 (14.8 $\times$)
DroidDetect-Base	Java (64/18)	0.161	0.003 (63.3 $\times$)	0.139	0.009 (15.4 $\times$)
DetectCodeGPT	Py (94/32)	0.687	0.280 (2.5 $\times$)	0.843	0.824 (1.0 $\times$)
DetectCodeGPT	Java (64/18)	0.645	0.244 (2.6 $\times$)	0.566	0.866 (0.7 $\times$, increase)

are illustrative only and support no confirmatory claim; we report them for transparency about what was attempted and why it did not scale, not as evidence for or against any detector’s stability.

5 Discussion

Static accuracy does not characterize post-transformation reliability, but this study’s results show something further and less obvious: neither does a low Induced Error Rate, by itself, characterize genuine stability. LLMSniffer’s near-majority-class (or, in Java, below-majority-class) accuracy on this corpus makes its low IER a likely artifact of a degenerate, near-constant, or even anti-correlated predictor rather than a sign of real robustness. DetectCodeGPT’s IER is not yet separable from its own re-scoring noise, measured directly via the identity-transformation control rather than assumed to be negligible. Only DroidDetect-Base’s instability survives both of these checks: it is a detector with real, high baseline accuracy (98–99%, though its dev-split validation carries an unresolved gap discussed in Section 6) whose formatting sensitivity is reproducible under deterministic inference, class-specific, and directionally asymmetric under a genuinely different, reverse intervention (58.0% vs. 5.2%; Section 4.1).

This asymmetry — more than the raw, and imperfectly length-controlled, edit-distance comparison discussed in Section 6 — is what best supports the interpretation that the detector has learned tidy formatting as a proxy signal for AI authorship, rather than genuinely modeling origin. CAST does not identify the underlying mechanism directly; that would require inspecting the detector’s internal representations or its training data, which is outside this paper’s scope. But the observable behavior — a large, reproducible, and one-sided sensitivity to a purely cosmetic, deterministic transformation — is consistent with a detector that has, whether intentionally or not, absorbed code style as a shortcut for origin during training.

This finding motivates reporting stability under transformation as a standard companion to accuracy for any detector intended for real-world use, and suggests canonicalization as a lightweight diagnostic worth running before deploying a detector, even though this paper’s own results show that the specific token-rejoining canonicalizer used here is not yet a validated mitigation — its own out-of-distribution effect means a practical fix likely requires a layout-preserving normalizer that keeps canonicalized code closer to a detector’s training distribution, rather than one that strips presentation information as aggressively as this study’s diagnostic canonicalizer does.

A verdict that cannot survive a random seed, or cannot survive a formatter being run on otherwise-unmodified code, is unfit for the single-verdict, high-stakes use this paper’s motivation describes. The practical implication for anyone deploying or relying on an AI-code detector is not simply “use a more accurate detector,” but “demand evidence of attribution stability, checked against both a baseline-accuracy floor and a re-scoring-noise floor, before trusting any individual verdict.”

For an instructor, hiring team, or compliance reviewer evaluating whether to adopt a specific detector, this paper’s results translate into a concrete pre-deployment checklist: (1) measure the detector’s own accuracy on a sample of your target corpus, not just its published number, since Section 3.4 shows these can diverge sharply; (2) run an identity-transformation control (Section 2.3) to establish a re-scoring noise floor before trusting any reported instability rate; (3) run the detector on the same file before and after an IDE’s default formatter, and treat any flip on human-authored code as a false-accusation risk specifically, not a generic error rate, since this paper’s asymmetry result suggests false accusations of human authors, not missed detections of AI authors, are where a formatter-sensitive detector fails; and (4) do not adopt a detector for a single-verdict, high-stakes decision until it has been checked against all three of the confounds this paper identifies (baseline accuracy, noise floor, and formatting sensitivity).

6 Threats to Validity

External validity. This evaluation covers two languages, one corpus, and three detectors; two additional targeted detectors could not be integrated because no checkpoint was released for either. The transformation operators used represent common tooling edits (an automatic formatter, a single rename, a single loop rewrite), not a sampled distribution of real developer edit histories, which may differ in composition and intensity.

Construct validity. LLMSniffer’s baseline accuracy on this corpus (67.0% Python, at its own majority-class baseline; 44.4% Java, below its majority-class baseline) means its low IER cannot be read as robustness evidence under any interpretation. Length-normalized $d_{\text{text}}/\text{len}$ is comparable (0.146 vs. 0.136) between flipped and unflipped human samples in the formatting-family analysis, so program length remains an unexcluded confound alongside the class-level asymmetry established in Section 4.1; the messify reverse check (Section 4.1) is also not intensity-matched with formatting, so it establishes directionality but not a precisely comparable effect size. DetectCodeGPT’s weak calibration limits its threshold-derived binary results to secondary status throughout, and its reported IER and $|\Delta|$ are upper bounds that include a comparably sized identity-control noise floor rather than a clean estimate of transformation-induced instability alone. Detector scores have incompatible scales across implementations (Section 2.2), and d_{ext} , d_{token} , and d_{AST} correlate only weakly with $|\Delta|$ even for the detector with the clearest instability signal, meaning distance is a useful but incomplete proxy for how much a detector’s output should be expected to move.

Internal validity. DroidDetect-Base’s architecture was reconstructed from its checkpoint in the absence of any public modeling code, and this reconstruction has not yet been compared numerically against the original authors’ own reported results. Its 87.5% dev-split accuracy’s 95% confidence interval, [69.0, 95.7], does not fully contain its 98–99% test-split accuracy reported in Section 4.1 — an unresolved gap that this paper’s single most important empirical finding rests entirely on, and that we flag explicitly rather than paper over. DroidDetect-Base’s IER throughout this paper uses the human-versus-AI collapse of its underlying four-class output, not raw four-class flips; of 365 transformed pairs examined for this collapse, 23 were intra-AI churn (between MACHINE_GENERATED and MACHINE_REFINED); of those, 16 were among baseline-correct pairs and were excluded from IER on the grounds that they do not change the human-versus-AI verdict. The canonicalizer used in Section 4.2 normalizes several distinct presentation properties simultaneously (whitespace, blank lines, comments, and quote style), so a positive result from it supports a claim about that combined category of variation, not about which specific property within it is responsible; canonicalized-input accuracy, now measured and reported directly rather than assumed, shows a real out-of-distribution effect for DroidDetect-Base that the observed movement reduction does not fully control for.

Corpus-composition validity. Droid’s fragment-heavy composition limits the deeper transformation families examined in Section 4.3 to 4.65% applicability, and limits differential-execution verification specifically to 4 of 35 successful instances; the corresponding per-detector cells are illustrative only, not confirmatory, and should not be read as evidence for or against stability under those deeper transformations.

Conclusion validity. Some cells in this study’s tables have small baseline-correct denominators, as low as $n=4$; Wilson 95% confidence intervals are reported throughout specifically for this reason, rather than relying on point estimates alone. Cohen’s d_z [21], reported in the accompanying artifact, is treated only as supplementary, directional-consistency evidence, never as a headline statistic, since it can overstate practically small movements once standardized by variance.

Table 4: Novelty positioning against the closest prior work.

Work	Post-gen. edit	Routine, non-adv.	AI-code detector	Cross-det.
Detector papers [2–5]	partial	partial	✓	–
CodeGPTSensor+ [6], Droid [1]	✓	– (adversarial)	✓	partial
Pordanesh et al. [11]	–	–	✓	✓
RoPGen [12]	✓	✓	– (human authorship)	–
Code-model robustness [7–10]	✓	✓	–	partial
CAST (this work)	✓	✓	✓	✓

7 Related Work

Measuring prediction change under fixed, semantics-preserving transformations is an established methodology for code models generally, and CAST does not claim novelty for that general idea. Rabin et al. [9] show that neural program models fail to generalize under a range of semantic-preserving program transformations. ReCode [10] benchmarks code-generation robustness under more than 30 similarly-styled operators applied to prompts and completions. Both establish, for code models broadly, exactly the kind of result CAST is built to produce for AI-code detectors specifically: that a model’s output can change substantially under an edit that does not change what the program does. CAST’s contribution is applying this established style of test to a different task and a different failure mode — attribution false-accusation, framed explicitly as a metamorphic relation [8], rather than degraded generation quality — and to a task (human-versus-AI attribution) where the practical stakes of an unstable verdict are qualitatively different from a worse code completion.

Table 4 positions CAST against the closest prior work along four dimensions: whether the work edits code post-generation at all, whether that edit is routine rather than adversarial, whether the work targets already-trained AI-code detectors specifically (as opposed to code models in general or human-authorship attribution), and whether it compares multiple detectors under identical edits. Detector papers themselves [2–5] report accuracy, occasionally under one perturbation, but do not stratify by transformation family or instrument edit distance. CodeGPTSensor+ [6] and Droid [1] share this paper’s underlying observation that renaming and refactoring can obscure AI origin, but address it through adversarial training or explicitly adversarial attack scenarios, not by measuring an already-trained, unmodified detector’s stability under a non-adversarial edit. Pordanesh et al. [11] vary the training set and prompting conditions used to *generate* the code under test, not the source code itself after generation. RoPGen [12] applies style transformation *during training* to harden a *human*-authorship attributor against stylistic evasion; CAST instead applies controlled transformation *at evaluation time*, against *already-trained*, unmodified AI-versus-human detectors, which is a materially different setting: RoPGen asks whether a detector can be trained to resist style manipulation, while CAST asks whether existing, deployed detectors already fail under style changes nobody intended as an attack. Feature-based work identifying whitespace as a particularly discriminative signal for AI-code detection [13] helps motivate, after the fact, why this paper’s finding that formatting sensitivity is so strongly presentation-driven for DroidDetect-Base is plausible rather than surprising.

8 Future Plans

This evaluation will be extended in four directions, prioritized by how directly they close the gaps identified in Section 6.

First, and highest priority, strengthening the validation of DroidDetect-Base’s reconstruction: comparing its outputs numerically against the original authors’ own reported results where those become available, and, if possible, contacting the original authors directly to resolve the dev/test accuracy gap identified in Section 3.4. Since this paper’s central finding depends entirely on this one reconstructed detector, closing this gap is the single highest-value next step.

Second, running an intensity-matched version of the reverse messify experiment (Section 4.1), calibrating the messiness transformation’s normalized edit size to match formatting’s on human code as closely as possible, so that the 58.0%/5.2% asymmetry can be reported as a properly matched effect-size comparison rather than only a directional one; alongside this, extending the identity-transformation noise-floor control from the current 40-sample dev-split subset to the full 199-program test set, and rescuing Detect-CodeGPT’s signal, where possible, with shared perturbation seeds across C_0 and $T(C_0)$ to reduce its measurement noise directly rather than only reporting it.

Third, replacing or supplementing the token-rejoining canonicalizer used in Section 4.2 with a layout-preserving normalizer, one that corrects specific presentation properties (indentation width, quote style, blank-line count) individually while leaving the code’s overall structure closer to what the detector was trained on, together with a component-level ablation (whitespace-only, comments-only, quote-style-only) to determine which specific presentation property drives DroidDetect-Base’s instability, rather than only the combined category this study currently reports.

Fourth, extending the corpus itself: a collection of complete, independently executable programs with test suites and chained, multi-step transformations mined from real commit histories, to scale beyond the 4.65% applicability of the deeper transformation families documented in Section 4.3, and to substitute a sampled distribution of real developer edits for the tooling-driven transformations used throughout this study. Longer term, this points toward a detector explicitly trained against CAST’s transformation families, to test directly whether attribution stability can be achieved by construction, rather than only diagnosed after the fact as this paper does.

9 Conclusion

This paper presented CAST, a controlled, distance-instrumented methodology for testing the attribution stability of AI-generated-code detectors under routine, non-adversarial source transformations. Applied to three detectors over 199 held-out Python and Java programs, its clearest finding is that a reconstructed but accuracy-validated detector (98–99% test accuracy, though its dev-split validation carries an unresolved gap discussed in Section 6) flips correct verdicts on genuinely human-written code to “AI” 58.0% of the time after routine reformatting, versus only 0.0–4.3% on actually AI-authored code, and only 5.2% in the reverse direction under a non-intensity-matched symmetry check — a pattern consistent with the detector having learned tidy formatting as a proxy for

AI authorship, and a genuine mislabel under the underlying corpus’s own construct definitions, not a defensible edge case. The other two detectors evaluated illustrate a separate, equally important methodological lesson: their apparently low instability is confounded, respectively, by baseline accuracy at or below chance and by re-scoring noise of comparable magnitude to the effect being measured, and neither can be read as genuine evidence of robustness without the accompanying controls this paper introduces. Static accuracy alone does not characterize a detector’s reliability under routine code evolution, and, this paper’s results suggest, neither does a low instability rate by itself — both require the kind of direct, controlled measurement CAST provides.

Artifact Availability Statement

An anonymized artifact accompanies this submission at <https://anonymous.4open.science/r/cast-isec2027-artifact-461E>, mirroring a GitHub repository scrubbed of author-identifying information for double-blind review. The artifact directly supports every quantitative claim in this paper and consists of:

- **Scripts** implementing the full CAST pipeline: manifest and transformation construction (formatting, lexical renaming, control-flow rewriting, and the deeper structural/semantic-rewrite families of Section 4.3), the three detector wrappers (LLMSniffer, the reconstructed DroidDetect-Base, and DetectCodeGPT), the canonicalization intervention, the identity-transformation noise-floor control (Section 2.3), the reverse “messify” experiment (Section 4.1), the human-vs-AI four-class-to-binary reclassification used throughout, the Fisher’s exact significance test reported in Section 4.1, and the table/figure generation scripts that produce every number and plot reported in the paper directly from frozen result files, with no manual editing step in between.
- **Frozen result artifacts:** JSONL score files and derived CSV tables for every experiment phase, including the full per-family breakdowns for all three detectors (Table 2 reports these in full; no result is hidden or held back), the per-origin-class breakdown underlying Table 1, the canonicalization before/after scores underlying Table 3 and Figure 2, and the identity-control and messify-experiment raw outputs.
- **Documentation:** a README describing how to regenerate every table and figure from the frozen artifacts (no GPU or model download required for this step), a `Dockerfile` and `requirements-reproduce.txt` for turnkey environment setup, pre-registration and results write-ups for each experimental phase, and an environment snapshot recording the software versions and hardware used to produce the original results.

Reproducing the tables and figures from the frozen artifacts requires no manual environment setup: a provided `Dockerfile` builds a container with pinned dependencies and regenerates every table and figure in this paper (`docker build`, then `docker run`), which we verified end-to-end while preparing this artifact; a `requirements-reproduce.txt` is also provided for reproducing the same tables/figures in a local Python environment without Docker. Reproducing the underlying detector scores from raw code, rather than from the frozen artifacts, requires the three detectors’

own dependencies (documented in the artifact) and, for DetectCodeGPT, GPU access, since it depends on CodeLlama-7B and CodeT5p-770M; this heavier path is intentionally kept outside the Docker image. Detector model checkpoints and cached Hugging Face model weights are not redistributed in the artifact, as they are large, third-party, and already publicly hosted at the citations given in Section 3.4; the artifact’s environment snapshot records their exact versions and provenance. We intend to submit this artifact for evaluation and are targeting the *Available*, *Functional*, and *Reusable* badges.

References

- [1] D. Orel, I. Paul, I. Gurevych, P. Nakov, “Droid: A Resource Suite for AI-Generated Code Detection,” *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- [2] M. L. Dihan, A. Muhtasim, “LLMSniffer: Detecting LLM-Generated Code via GraphCodeBERT and Supervised Contrastive Learning,” arXiv:2604.16058, 2026.
- [3] P. T. Nguyen, J. Di Rocco, C. Di Sipio, R. Rubei, D. Di Ruscio, M. Di Penta, “GPT-Sniffer: A CodeBERT-based classifier to detect source code written by ChatGPT,” *Journal of Systems and Software*, 2024.
- [4] Y. Shi, H. Zhang, C. Wan, X. Gu, “Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers,” *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [5] X. Xu, C. Ni, X. Guo, S. Liu, X. Wang, K. Liu, X. Yang, “Distinguishing LLM-generated from Human-written Code by Contrastive Learning,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 91, 2025.
- [6] X. Yin, X. Li, C. Ni, X. Xu, X. Yang, “Detecting LLM-generated Code with Subtle Modification by Adversarial Training,” arXiv:2507.13123, 2025.
- [7] T. Suresh, S. Ugare, G. Singh, S. Misailovic, “Is the Watermarking of LLM-Generated Code Robust?” arXiv:2403.17983, 2024.
- [8] A. Asgari, M. de Koning, P. Derakhshanfar, A. Panichella, “Metamorphic Testing of Deep Code Models: A Systematic Literature Review,” arXiv:2507.22610, 2025.
- [9] M. R. I. Rabin, N. D. Q. Bui, K. Wang, Y. Yu, L. Jiang, M. A. Alipour, “On the Generalizability of Neural Program Models with respect to Semantic-Preserving Program Transformations,” *Journal of Systems and Software*, 2021.
- [10] S. Wang, Z. Li, H. Qian, C. Yang, Z. Wang, M. Shang, V. Kumar, S. Tan, B. Ray, P. Bhatia, R. Nallapati, M. K. Ramanathan, D. Roth, B. Xiang, “ReCode: Robustness Evaluation of Code Generation Models,” *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [11] S. Pordanesh, S. Bukhari, B. Tan, L. De Carli, “Hiding in Plain Sight: On the Robustness of AI-Generated Code Detection,” *Proceedings of the 22nd Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2025.
- [12] Z. Li, G. Q. Chen, C. Chen, Y. Zou, S. Xu, “RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation,” *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022.
- [13] S. M. H. Nirob, S. Ehsan, M. Rahman, S. Haque, “Whitespaces Don’t Lie: Feature-Driven and Embedding-Based Approaches for Detecting Machine-Generated Code,” arXiv:2601.19264, 2026.
- [14] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, C. Finn, “DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature,” arXiv:2301.11305, 2023.
- [15] J. Su, T. Y. Zhuo, D. Wang, P. Nakov, “DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text,” *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- [16] B. Rozière, J. Gehring, F. Gloeckle, et al., “Code Llama: Open Foundation Models for Code,” arXiv:2308.12950, 2023.
- [17] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, S. C. H. Hoi, “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” arXiv:2305.07922, 2023.
- [18] B. Warner, A. Chaffin, B. Clavié, et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” arXiv:2412.13663, 2024.
- [19] D. Guo, S. Ren, S. Lu, et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” arXiv:2009.08366, 2020.
- [20] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [21] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.